The Agent Stack - Part 2: Foundation Infrastructure, Models, and Inference

Why the stack starts lower than most people think

VINOTH GOVINDARAJAN
APR 06, 2026

An agent can look fine right up to the moment you give it a real request: a long PDF, one tool call, and a strict JSON answer. Then latency jumps, the tool loop gets brittle, and the system suddenly looks a lot less intelligent than it did in the demo.

Nothing magical happened. You just hit the foundation.

This post is about the contract every agent system inherits before it gets a control plane, memory, or approvals. My read is that the cleanest lower-layer model starts with two layers, infrastructure substrate and model engine, then splits the model engine into model asset, serving system, and interaction contract.

This is not a model roundup. It is not an infra listicle. It is a systems model for the lower layers, the part of the stack that decides what the rest of the system is allowed to assume.

Before you have an agent, you already have semantics

By the time a model sees a token, a lot has already been decided. Where the work runs. How it gets scheduled. What survives a retry. Whether a dropped subscriber loses the event forever or can catch up later. Whether a write is visible immediately or only eventually. Whether one tenant can starve another.

Those are not background details. They are the operating semantics the rest of the stack inherits.

Kubernetes treats GPUs as schedulable resources. Redis is blunt about the difference between Pub/Sub and Streams. PostgreSQL's MVCC and WAL are about isolation and integrity under concurrency, not just storage. S3's consistency model changes what a reader can safely assume after a write.

The useful question is not "what infra are you on?" It is "what semantics do you inherit?"

If that still feels abstract, think about the first failure you hit when a notebook demo becomes a real service. It is usually not "the model got worse." It is a lower-layer problem: retries, ordering, stale state, dropped events, or latency. A typing indicator

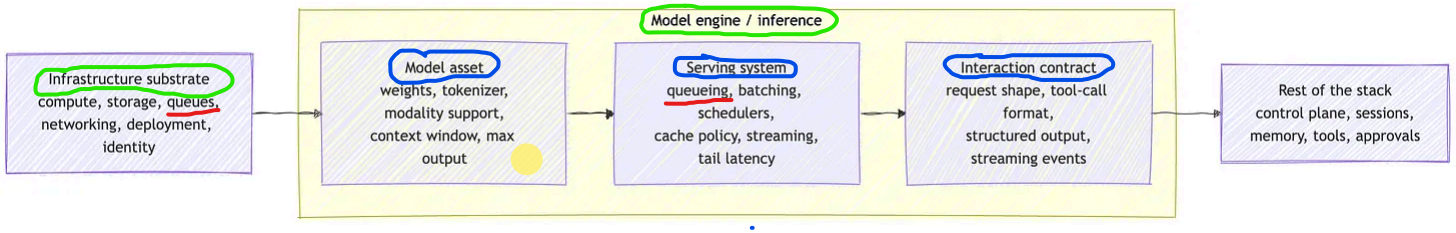
can tolerate a lost event. A durable workflow step cannot. A best-effort cache is fine for speed. It is a bad source of truth.

That is why “infrastructure substrate” is a useful term here. It keeps the focus on delivery, consistency, isolation, and failure behavior, not product names.

Stop treating the model layer as one blob

Most writing about agent systems jumps straight from infrastructure to “the model.” That makes the middle disappear.

If you remember one picture from this post, make it this one.



The point is not to create extra labels. It is to make the lower-layer boundaries visible again. What people often call “the model layer” is really three different things, and each one constrains the stack above it in a different way.

The first piece is the model asset. That is the weights, tokenizer, modality support, context window, max output, and whatever capability envelope the provider exposes. Those are real constraints. Providers publish them because upper layers depend on them.

The second piece is the serving system. This is where a model asset becomes an actual service with queueing, schedulers, cache policy, batching, streaming, and a real tail-latency profile. Across vLLM, TGI, TensorRT-LLM, and SGLang, the same operational vocabulary keeps showing up. That repetition is the clue. A large share of system behavior comes from runtime and cache management, not just from the weights.

The third piece is the interaction contract. OpenAI's Responses API is stateful and tool-aware. Anthropic separates client-executed tools from server tools. Gemini now spans `generateContent`, a beta Interactions API, and a Live API for realtime sessions. Upper layers do not talk to "the model" in the abstract. They talk to one of these contracts.

> The stack starts lower than most people think because the system inherits semantics before it inherits "agent behavior."

This is where a lot of agent talk gets slippery. People say "the model can use tools" when what they really mean is "the API exposes a tool-call format and the application agrees to honor it." interaction contract?

The useful boundaries are the real story

Once you split the lower layers cleanly, the category mistakes get easier to see.

Long context is useful, but it is still request-time state. The model only sees what you assemble for this turn. If that state needs to persist across turns or sessions, some other system has to own it, store it, and decide when to re-inject it.

> A context window is a working set, not memory. so which layer?

That distinction matters because people often talk as if a larger context window removes the need for state management. It helps, but it does not replace it.

A tool call is a proposal, not execution. The model can suggest an action and shape its arguments. Your code still executes it, or refuses to.

Structured output is a constraint on shape, not a guarantee of truth or policy compliance. A schema-valid object can still be wrong. A well-formed tool call can still be unauthorized.

Caching is another place people collapse layers together. OpenAI prompt caching, Anthropic prompt caching, Gemini context caching, and serving-layer prefix or KV reuse solve related problems at different layers. Same family of idea, different mechanism, different lever.

An OpenAI-compatible endpoint can make migration easier. It can hide differences at the client boundary, but it does not tell you much about scheduler behavior, cache reuse, or latency under load. That is useful compatibility, not equivalence.

> Compatible is not equivalent. A common API can smooth over client differences without making the systems underneath behave the same.

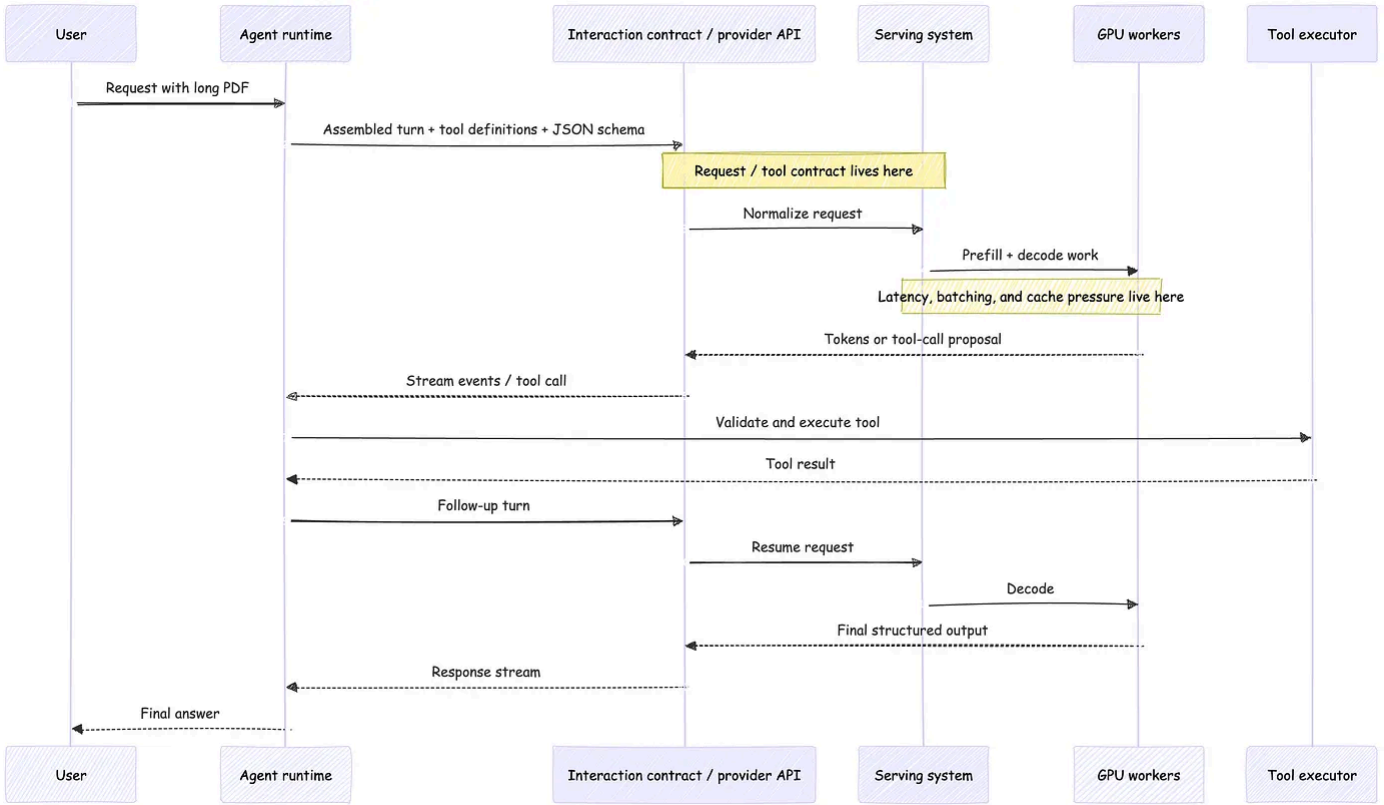
A long PDF is not just more context

A long PDF, one tool call, and a strict JSON answer is a good stress test because it crosses all three parts of the model layer at once.

The model asset sets the token budget and modality limits. The serving system pays the prefill cost, manages cache pressure, and determines how gracefully that large request coexists with everything else on the box. The interaction contract decides how the document is packaged, how the tool call is represented, how streaming behaves, and what “strict JSON” actually means on the wire.

That is why a system can look stable in short chat and suddenly feel brittle on a real request shape. The issue is not that the agent became mysterious. The happy path just stopped hiding the lower layers.

In practice, long inputs usually mean more prefill work, more cache pressure, more bytes in motion, and a wider blast radius if you have to retry around tool execution. That is what the foundation feels like from above.



This is why the rest of the stack moves upward from here

Lower-layer constraints do not stay in the lower layers.

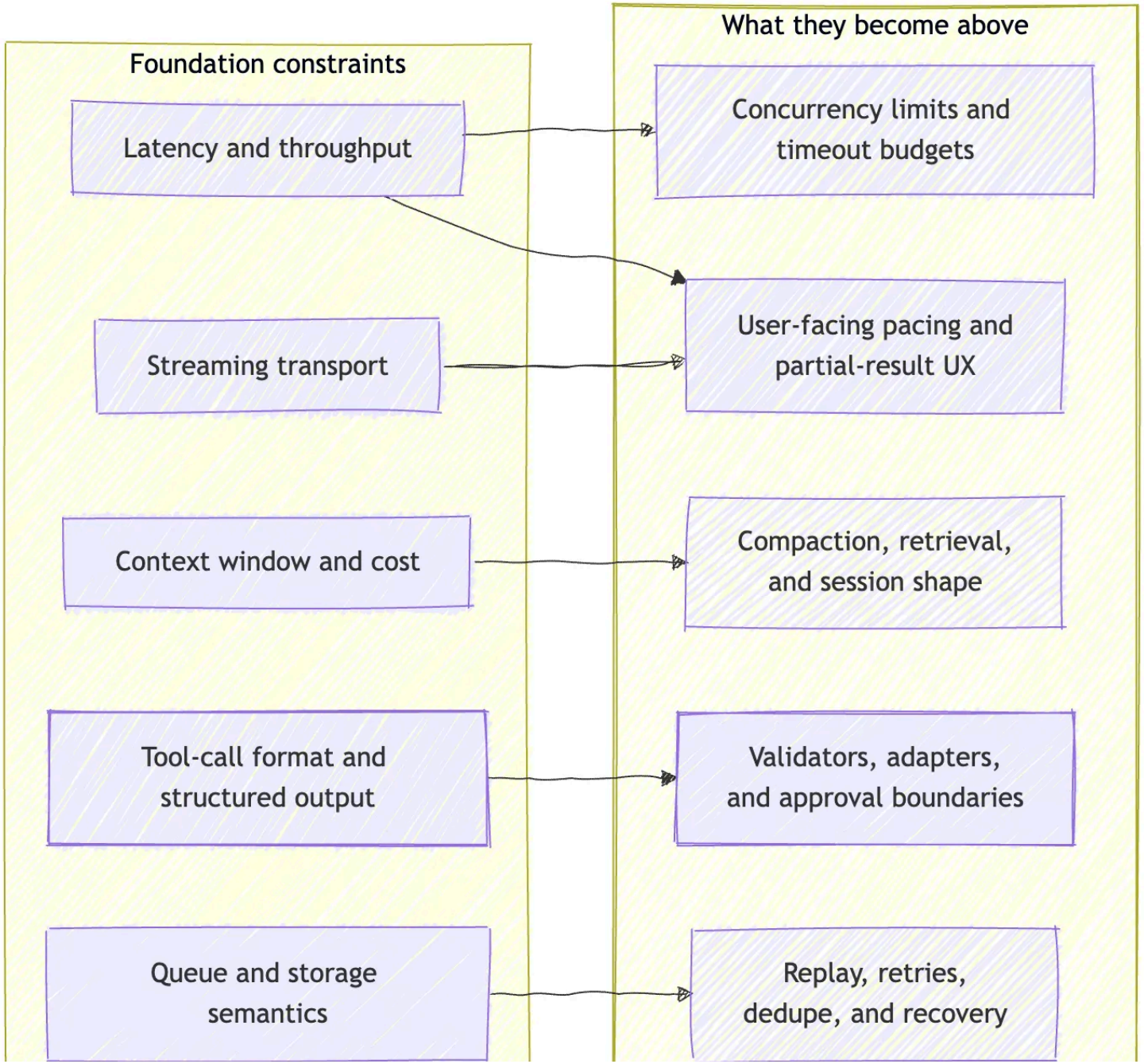
Latency and throughput show up later as concurrency limits, timeout budgets, and user-facing pacing. Streaming helps because it exposes progress earlier. It does not remove queueing, prefill, or decode cost.

Context window and cost show up later as compaction, retrieval policy, and session shape. Long context can simplify some workloads. It does not remove the need to decide what state lives where, or how much of it you can afford to keep hot.

Tool-call format and structured output show up later as validators, adapters, and approval boundaries. A schema-valid call can still be unsafe to execute. That is why runtimes and control planes exist at all.

Queue and storage semantics show up the moment something fails. If the transport is at-most-once, recovery looks different than it does on an append-only log. If the state store gives you transaction isolation, mutation looks different than it does on a best-effort cache.

By the time you get to control planes and sessions in Part 3, these budgets and semantics are already in the room.


Foundation constraints
What they become above
Latency and throughput
Concurrency limits and timeout budgets
Streaming transport
User-facing pacing and partial-result UX
Context window and cost
Compaction, retrieval, and session shape
Tool-call format and structured output
Validators, adapters, and approval boundaries
Queue and storage semantics
Replay, retries, dedupe, and recovery

The rest of the stack is downstream of these budgets, not separate from them.

Failure modes worth caring about

1. Using a fire-and-forget transport for durable work. Real-time signaling and durable workflow steps are not the same problem.
2. Calling long context “memory.” A larger working set helps, but it does not solve state ownership across turns or sessions.
3. Mistaking structured output for authority. JSON schema compliance solves packaging problems. It does not solve validation, permissions, or execution safety.
4. Mistaking API compatibility for system equivalence. Client portability is useful. It is not the same thing as semantic equivalence under load.
5. Treating all caching as one thing. Provider caching, explicit context caching, and serving-layer prefix reuse are not interchangeable.

Builder checklist

If I were pressure-testing an agent stack at this layer, these are the controls I would actually care about:

- Write down the delivery and consistency assumptions your runtime depends on.
- Separate model choice from serving choice and from API-contract choice.
- Benchmark the real request shape, not the toy prompt.
- Treat tool calls and structured outputs as untrusted input until your code validates them.
- Be precise when you say “cache.” Name the layer.
- Carry context, latency, and cost budgets upward into session and control-plane design.
- Do not let “compatible” stand in for “equivalent.”

Recap

The point of Part 2 is not to make the stack more elaborate. It is to make the later parts less fuzzy.

Once you see the lower layers clearly, Part 3 has something solid to stand on. A control plane is not just the thing that orchestrates. It is the layer that has to make these budgets, semantics, and boundaries usable across runs and sessions.

What comes next

In Part 3, I'll move up one layer and look at control planes, sessions, and state ownership, the layer that has to make these budgets and semantics usable across runs.

That is where the system decides what a run is, which state belongs to it, and how continuity is preserved without confusing it with memory or authorization.

New here? Start with Part 1 for the full stack map. Subscribe if you want the rest of the series as it publishes.

References / further reading

- OpenAI Responses API, Models, Structured Outputs, Using tools, Prompt caching, and Streaming responses
- Anthropic Models overview, Tool use overview, How tool use works, Streaming messages, Prompt caching, and Increase output consistency
- Gemini Models, Generating content, Function calling, Structured outputs, Long context, Context caching, Interactions API, and Live API
- vLLM OpenAI-compatible server, Text Generation Inference (TGI, now in maintenance mode), TensorRT-LLM overview, and SGLang architecture overview

- Kubernetes GPU scheduling, Kubernetes multi-tenancy, Redis Pub/Sub, Redis Streams, PostgreSQL MVCC, PostgreSQL WAL, and Amazon S3 strong consistency

21 Likes · 4 Restacks

Discussion about this post

Comments Restacks

Write a comment...

Pawel Jozefiak 7 abr

Liked by Vinoth Govindarajan

The point about the stack starting lower than most people think is accurate. I spent months focused on prompts and instruction files before realizing the serving layer was the source of inconsistency. Same model, same prompt, different caching behavior = different results. For anyone starting out: the interaction contract section here is worth reading slowly.

The long PDF with tool calls example you used is exactly the kind of edge case that breaks beginners. Simple chat demos hide the brittleness completely. It only shows up under real load with complex tool use - by which point you've already built half your system.

LIKE (1) REPLY SHARE

1 reply by Vinoth Govindarajan

1 more comment...